

DOCUMENT DE RÉFÉRENCE TECHNIQUE

Plateforme Diwan

Choix techniques, méthodes et organisation du travail pour publier les khassaïdes de Cheikh Ahmadou Bamba sur le web et sur mobile

Élément	Détail
Projet	Numérisation, vocalisation et publication des sept diwans
Technologies	PostgreSQL · Django et Django REST Framework · Angular et Bootstrap · Flutter · Render et Neon
Destinataires	L'équipe du projet et ses encadrants
Version	3.0, septembre 2026 (ajoute la mise en ligne et le travail à deux)

Sommaire

1. Présentation du projet.....	4
1.1 Objectif.....	4
1.2 Le corpus.....	4
1.3 Ce que la plateforme offre au lecteur.....	4
1.4 Architecture générale.....	5
2. Choix technologiques et justifications.....	5
2.1 PostgreSQL plutôt que MySQL.....	5
2.2 Django plutôt que Node.js ou Spring Boot.....	6
2.3 Angular avec rendu côté serveur.....	6
2.4 Une API entre la base et les applications.....	6
3. Organisation du dépôt.....	7
4. La base de données.....	7
4.1 Principe : Diwan → Poème → Ligne.....	7
4.2 Les parties d'un poème.....	7
4.3 Le nom du poème.....	8
4.4 L'acrostiche.....	8
4.5 Les tables.....	8
4.6 Les contraintes d'intégrité.....	10
4.7 Texte affiché et texte de recherche.....	10
5. Méthode : du fichier Word à la base de données.....	10
5.1 Préparer le fichier d'un poème.....	10
5.2 Lecture automatique et contrôles.....	11
5.3 L'outil web « Poem to JSON ».....	11
5.4 Importer dans la base et publier.....	11
6. La transcription en caractères latins.....	12
6.1 Pourquoi elle est générée automatiquement.....	12
6.2 La convention locale retenue.....	12
6.3 Les règles de prononciation.....	12
6.4 Stockage et corrections.....	13
7. L'API REST.....	13
8. Le site web (Angular).....	14
8.1 Les pages.....	14
8.2 Le rendu côté serveur.....	14
8.3 Identité visuelle.....	14
8.4 Organisation du code.....	15
8.5 Détails de réalisation.....	15
9. Installer et lancer le projet.....	15
9.1 Prérequis.....	15
9.2 Le backend.....	16
9.3 Le site web.....	16
9.4 Windows, macOS et Linux.....	16
10. Travailler à deux avec Git et GitHub.....	16
10.1 Principe.....	16
10.2 Création du dépôt (responsable backend, une seule fois).....	17
10.3 Première installation (responsable frontend).....	18
10.4 Le cycle de travail quotidien.....	19
10.5 Règles pour éviter les conflits.....	19
10.6 Résoudre un conflit.....	19
10.7 Côté backend : faire passer une modification dans le dépôt.....	20
10.8 Côté frontend : faire passer une modification dans le dépôt.....	20
11. L'application mobile Flutter.....	21
11.1 Principe.....	21
11.2 Création dans le dépôt.....	21
11.3 Organisation du code.....	22
11.4 Premier appel à l'API.....	23
11.5 Adresse de l'API pendant le développement.....	23
11.6 Plus tard.....	24
12. La plateforme en ligne.....	24
12.1 Les deux services et la base.....	24

12.2 Le service de l'API (Django).....	3
12.3 Le service du site (Angular).....	3
12.4 Alimenter la base en ligne.....	3
12.5 Sécurité et confidentialité.....	3
12.6 Problèmes rencontrés et leur cause.....	3
13. Prochaines étapes.....	3

1. Présentation du projet

1.1 Objectif

Le projet Diwan rassemble l'œuvre poétique de Cheikh Ahmadou Bamba contenue dans sept diwans, dans une édition numérique complète et entièrement vocalisée, publiée sur un site web et sur une application mobile. Chaque khassida y possède sa propre page : texte vocalisé, nom du poème, lettres de l'acrostiche mises en évidence, transcription en caractères latins et, à terme, fichier PDF à télécharger.

Ce document décrit les choix techniques retenus, les raisons de chacun de ces choix et les méthodes appliquées jusqu'ici, jusqu'à la mise en ligne de la plateforme. Il sert de référence commune à l'équipe (partie serveur, site web, application mobile) et aux encadrants du projet. Les parties 10 et 12 expliquent, pour chaque rôle, comment faire passer son travail dans le dépôt commun et sur la plateforme en ligne.

1.2 Le corpus

Les chiffres ci-dessous ont été calculés automatiquement sur le texte intégral des sept diwans (fichier `diwan_corpus_complet.docx`). Les limites entre les diwans ont été vérifiées et confirmées.

N°	Diwan	Abyat	Mots
1	ديوان القرآنية	5 588	65 700
2	ديوان الأمداح النبوية	6 059	71 121
3	مراقي الأمن والسعادة	3 840	44 631
4	الفيوضات الربانية بالأعوام والشهور	5 523	67 682
5	القصائد المطرزة بغير الآيات القرآنية	5 387	69 448
6	الفيوضات الربانية في الذكر والشكر والتحدث بالنعم الإلهية	4 857	57 324
7	الفلك المشحون	8 994	85 164
	Total	40 248	461 070

Le corpus compte également 81 561 hémistiches et 1 455 pages. Le nombre de mots (461 070) est inférieur à celui qu'affiche Word (480 557) parce que Word compte aussi les barres | qui séparent les hémistiches ; seuls les mots arabes sont comptés ici.

1.3 Ce que la plateforme offre au lecteur

- Une page par khassida, avec son texte vocalisé bayt par bayt et sous le nom que lui a donné son auteur.
- La transcription en caractères latins de chaque ligne, pour lire et réciter sans connaître l'écriture arabe.
- La recherche dans les noms des poèmes puis dans les vers, même sans taper les harakat.
- Des classements par diwan et, à venir, par mois de l'année hégirienne, par événement et par jour.
- Des fichiers PDF pour chaque poème, chaque diwan et le corpus entier (à venir).

1.4 Architecture générale

Toutes les données passent par un seul chemin. La base de données est la source unique de vérité : le site et l'application ne lisent jamais les fichiers Word.

```

Poème relu (.docx, un poème par fichier)
  | lecture automatique : parsing.py
  ▼
JSON du poème —(outil web « Poem to JSON » pour vérifier)
  | import : page d'administration ou commande import_poems
  ▼
PostgreSQL (Diwan → Poème → Ligne → Transcription)
  |
  ▼
API REST Django /api/v1/... (lecture seule, poèmes publiés uniquement)
  ├── Site web Angular (rendu côté serveur)
  └── Application Flutter (Android / iOS)

```

2. Choix technologiques et justifications

Chaque couche du projet utilise l'outil le mieux adapté à son rôle. Le tableau résume les choix ; les paragraphes qui suivent les justifient.

Couche	Technologie	Rôle
Base de données	PostgreSQL 16	Stocke les diwans, les poèmes, chaque ligne et sa transcription
Serveur	Python 3.12, Django 6, Django REST Framework	API, administration, import des poèmes, transcription
Site web	Angular 21, Bootstrap 5, rendu côté serveur (SSR)	Pages d'accueil, des diwans et des khassaïdes
Application mobile	Flutter (Dart)	Mêmes contenus que le site, sur Android et iOS
Outils	Flask, python-docx, WhiteNoise	Outil « Poem to JSON », lecture des .docx, fichiers de style de l'administration
Travail en équipe	Git et GitHub (un seul dépôt)	Historique du code, branches, relecture par pull request

2.1 PostgreSQL plutôt que MySQL

- **Tableaux natifs.** Un bayt est rangé comme un tableau d'hémistiches (`["sadr", "ajz"]`, ou quatre éléments). PostgreSQL gère ce type directement (ArrayField dans Django), sans table supplémentaire.
- **Recherche dans l'arabe.** L'extension `pg_trgm` crée un index par trigrammes sur le texte normalisé : la recherche d'un mot à l'intérieur de 40 000 abyat reste rapide, même quand le mot est attaché à un préfixe (و, ل, ب).
- **Contraintes de vérification.** La base refuse elle-même les données incohérentes, par exemple un bayt placé dans la khatima ou deux lignes « nom » dans un poème.
- **Gratuit, fiable, très utilisé en production** et parfaitement pris en charge par Django.

2.2 Django plutôt que Node.js ou Spring Boot

- **Un seul langage pour tout le traitement du texte.** La lecture des fichiers Word (python-docx), la normalisation de l'arabe, la vérification de l'acrostiche et la transcription sont écrites en Python ; le serveur les utilise directement.
- **Une administration fournie.** Django génère l'interface d'administration dans laquelle on importe les poèmes, on corrige une transcription ou on publie un poème, sans écrire d'interface supplémentaire.
- **Des migrations.** Chaque changement de structure de la base est un fichier versionné avec le code : les deux membres de l'équipe obtiennent la même base avec `python manage.py migrate`.
- **Django REST Framework** fournit l'API lue par le site et l'application.

2.3 Angular avec rendu côté serveur

- **Composants et TypeScript.** Le site est découpé en composants réutilisables (en-tête, bayt, fil d'Ariane) et le typage vérifie la forme des données reçues de l'API.
- **Rendu côté serveur (SSR).** Chaque page est construite sur le serveur à chaque visite : elle arrive complète, avec son texte arabe. Les moteurs de recherche indexent donc chaque khassida, et un poème importé apparaît immédiatement, sans reconstruire le site.
- **Bootstrap 5**, limité aux modules utilisés (grille, tableaux, boutons, utilitaires) et personnalisé aux couleurs du projet, pour garder un site léger.

2.4 Une API entre la base et les applications

Ni le site ni l'application mobile ne se connectent directement à PostgreSQL. Ils interrogent l'API du serveur. Ce choix protège la base (aucun mot de passe dans une application installée sur un téléphone), place les règles à un seul endroit (seuls les poèmes publiés sont visibles) et permet au site et à l'application d'afficher exactement les mêmes données.

3. Organisation du dépôt

Tout le projet vit dans un seul dépôt Git, avec un dossier par partie. Chaque membre de l'équipe travaille principalement dans son dossier.

```

diwan-platform/
├── backend/                Django : base de données, API, administration, import
│   ├── manage.py
│   ├── requirements.txt
│   ├── config/            réglages (settings.py) et adresses (urls.py)
│   ├── apps/corpus/       le corpus
│   │   ├── models.py     les tables de la base
│   │   ├── parsing.py    lecture d'un poème .docx, contrôles, JSON
│   │   ├── arabic.py     normalisation de l'arabe, lettres d'acrostiche
│   │   ├── transcription.py transcription en caractères latins
│   │   ├── importing.py  enregistrement d'un poème dans la base
│   │   ├── admin.py      administration et page « Import poems »
│   │   ├── api/          API REST : serializers.py, views.py, urls.py
│   │   ├── migrations/   historique de la structure de la base
│   │   └── management/commands/ import_poems.py, seed_diwans.py
│   ├── apps/exports/      fichiers PDF et DOCX (à venir)
│   └── tools/poem_json_web/ outil web « Poem to JSON »
├── frontend/             Angular : le site web
├── mobile/               Flutter : l'application (à créer)
├── corpus-json/          poèmes relus au format JSON (diwan-01/D01K08.json ...)
└── docs/                 ce document, notes sur l'API

```

4. La base de données

4.1 Principe : Diwan → Poème → Ligne

Chaque paragraphe d'un poème devient une ligne de la table `Line`, dans l'ordre de lecture, plutôt qu'un seul bloc de texte par poème. Ce choix permet :

- d'afficher n'importe quelle mise en page (deux ou quatre hémistiches) sur le site, sur mobile et en PDF ;
- de créer un lien vers un bayt précis, par exemple `/diwans/diwan-01/008#bayt-12` ;
- de chercher bayt par bayt et d'afficher le vers exact trouvé ;
- de rattacher plus tard d'autres informations à chaque bayt (traduction, i'rab, audio).

4.2 Les parties d'un poème

Chaque ligne indique la partie du poème à laquelle elle appartient (champ `section`) :

Partie	Contenu	Comment elle est reconnue
muqaddima	Le texte d'ouverture : basmala, salawat, dédicace	Les textes placés avant les abyat
title	Le nom du poème donné par l'auteur	Le deuxième texte, sur une seule ligne, juste avant les abyat
matn	Les abyat	Les lignes qui contiennent la barre
khatima	Le texte de clôture	Les textes placés après le dernier bayt

4.3 Le nom du poème

Le nom est l'information centrale de chaque poème : il sert à l'affichage, à la recherche et, plus tard, aux classements par mois, événement et jour. La règle retenue est la suivante :

Avant les abyat	Nom du poème	Champ title_source
Deux textes ou plus	Le deuxième texte, sur une seule ligne (acrostiche ou non)	name_line
Un seul texte	Le sadr du premier bayt ; le texte reste la muqaddima	first_sadr
Aucun texte	Le sadr du premier bayt	first_sadr

Exemples :

Poème	Nom	Acrostiche
D01K02	هَذِهِ أَسْمَاءُ الشُّوَرِ	Non
D01K08	أَعُوذُ بِاللَّهِ مِنَ الشَّيْطَانِ الرَّجِيمِ وَقَدْ أَعَادَنِي مِنْهُ	Oui, 100 %

4.4 L'acrostiche

Plus de 90 % des poèmes du Cheikh sont des acrostiches : les premières lettres des abyat, lues de haut en bas, forment une phrase (un verset coranique, le nom d'un mois comme Ramadan, ou une phrase choisie par l'auteur). Quand un poème possède un nom, le programme vérifie si ce nom est aussi un acrostiche.

- **Vérification souple.** Le nom est accepté comme acrostiche si au moins la moitié de ses lettres se retrouvent, dans l'ordre, au début des abyat d'ouverture. Certains poèmes répètent l'acrostiche trois fois puis continuent avec d'autres acrostiches : leur début suffit.
- **Lettres équivalentes.** Les formes de la hamza (أ | إ | ؤ) comptent comme ا, comme ة, ي comme ه, et le mot الله est compté avec son alif long (الله), comme le fait l'auteur.
- **Ce qui est enregistré :** `is_acrostic` (oui/non) et `acrostic_match` (pourcentage de lettres retrouvées). Sur le site, le nom et la première lettre de chaque bayt s'affichent alors en rouge.
- Un nom qui n'est pas un acrostiche est normal et ne produit aucun avertissement.

4.5 Les tables

Diwan

Champ	Type	Rôle
number, slug	entier, texte unique	1 à 7 ; « diwan-01 » dans les adresses
title, title_plain	texte	Titre en arabe ; version normalisée pour la recherche
corpus_abyat, corpus_hemistichs, corpus_words	entiers	Chiffres du texte intégral, affichés même avant l'import de tous les poèmes
status	brouillon / relecture / publié	Visibilité

Poem (khassida)

Champ	Type	Rôle
diwan, number, code, slug	lien, entier, textes	Diwan 1, poème 8, « D01K08 », « 008 »
title, title_source	texte, choix	Nom du poème et son origine (ligne du nom ou premier sadr)
incipit, title_plain	textes	Premier sadr ; nom normalisé pour la recherche (index par trigrammes)
is_acrostic, acrostic_match	booléen, pourcentage	Le nom est-il aussi un acrostiche, et à quel point
bayt_count, hemistichs_per_bayt	entiers	Nombre d'abyat ; 2 ou 4 hémistiches
has_open_flags	booléen	Mots signalés en rouge encore présents
status	brouillon / relecture / publié	Seuls les poèmes publiés sont visibles sur le site
content_hash, source_file	empreinte, texte	Détecte un texte modifié ; dernier fichier importé

Line (une ligne du poème)

Champ	Type	Rôle
poem, position	lien, entier	Poème et ordre de lecture
section, kind	choix	muqaddima / title / matn / khatima ; prose / title / bayt
bayt_number	entier ou vide	Numéro du bayt (vide pour la prose)
hemistichs	tableau de textes	Texte vocalisé : 2 ou 4 hémistiches, ou 1 élément pour la prose
text_plain	texte + index trigrammes	Texte sans vocalisation, normalisé, pour la recherche

LineTranscription

Champ	Type	Rôle
line, style	lien, choix	Ligne transcrite ; style « local » (anglais plus tard)
parts	tableau de textes	Une transcription par hémistiche
is_manual, needs_review	booléens	Corrigée à la main ; à revoir car l'arabe a changé

Export (préparé pour les PDF)

Champ	Type	Rôle
scope, poem, diwan	choix, liens	Fichier d'un poème, d'un diwan ou du corpus
format, file, source_hash	pdf/docx, fichier, empreinte	Le fichier et le texte à partir duquel il a été produit

4.6 Les contraintes d'intégrité

Ces règles sont inscrites dans PostgreSQL. Même une erreur de programme ne peut pas les enfreindre :

- un seul poème par numéro dans un diwan, un seul code « D01K08 » dans tout le corpus ;
- une seule ligne par position dans un poème, un seul numéro de bayt par poème ;
- un bayt se trouve toujours dans le matn, avec un numéro et 2 ou 4 hémistiches ;
- une seule ligne « nom » par poème, et uniquement dans la partie `title` ;
- une seule transcription par ligne et par style.

4.7 Texte affiché et texte de recherche

Chaque ligne est gardée deux fois : le texte vocalisé pour l'affichage, et une version normalisée pour la recherche. La même fonction de normalisation est appliquée au texte importé et au mot recherché, si bien qu'un lecteur qui tape القرآن sans harakat trouve القرآن tel qu'il est écrit dans le corpus.

Écrit ou tapé	Après normalisation	Ce qui a été unifié
الْقُرْآنِ	القران	Vocalisation supprimée, madda
القرءان	القران	Graphie du corpus + ء
إِلَى	الي	Hamza sous l'alif, alif maqsura
رَحْمَةً	رحمه	Tanwin, ta marbuta

5. Méthode : du fichier Word à la base de données

5.1 Préparer le fichier d'un poème

Après la vocalisation et la relecture, chaque khassida est enregistrée dans son propre fichier Word, selon ces conventions :

- **Un poème par fichier**, dont le nom commence par son code : D01K08_أعوذ بالله... .docx (diwan 1, khassida 8).
- **Un bayt par paragraphe**, les hémistiches séparés par | (trois barres pour les poèmes à quatre hémistiches).
- **Le nom du poème**, s'il existe, sur une seule ligne juste avant les abyat, après le texte d'ouverture.
- **Les mots incertains** en rouge pendant la relecture. Un poème qui garde des mots rouges n'est pas publié.

5.2 Lecture automatique et contrôles

Le programme `parsing.py` lit le fichier, reconnaît les parties du poème, détermine son nom, vérifie l'acrostiche, produit la transcription et effectue les contrôles suivants :

Contrôle	Exemple	Conséquence
Lettres en rouge	Mot incertain non corrigé	Drapeau : le poème reste en brouillon
Haraka seule restée rouge	Kasra rouge sur un mot noir (« هَات »)	Rappel à corriger avant impression, pas un drapeau
Espace en rouge	Espace après un mot recoloré	Ignoré, invisible
Haraka sans lettre	« عَتَاءٌ جَنَّة »	Avertissement
Deux voyelles sur une lettre	« حَطَا », « يُتِلُّه »	Avertissement
Nombre d'hémistiches	Un bayt avec 3 hémistiches	Avertissement
Plus de deux textes avant les abyat	Texte d'ouverture en plusieurs lignes	Demande de confirmation du nom

Le programme calcule aussi une **empreinte** (`content_hash`) du texte arabe du poème : elle permet de savoir, lors d'un nouvel import, si le texte a réellement changé.

5.3 L'outil web « Poem to JSON »

Pour vérifier un poème avant de l'importer, l'outil web local affiche le poème mis en page, ses avertissements et le JSON produit. On peut y déposer plusieurs fichiers .docx ou coller un poème.

```
cd backend
python tools/poem_json_web/app.py # puis ouvrir http://127.0.0.1:5000
```

Il permet de copier ou de télécharger le JSON, de télécharger tous les poèmes en .zip, ou de les enregistrer directement dans `corpus-json/diwan-01/`.

5.4 Importer dans la base et publier

Deux façons d'importer, qui utilisent exactement le même code (`importing.py`) :

- **Dans l'administration Django** : Poems > **Import poems**, puis choisir un ou plusieurs fichiers .docx ou .json. Un tableau affiche le résultat et les avertissements de chaque poème.
- **En ligne de commande** :

```
# une fois : créer les 7 diwans et leurs chiffres
python manage.py seed_diwans

# importer des dossiers ou des fichiers .json / .docx
python manage.py import_poems ../corpus-json/diwan-01/

# vérifier sans rien enregistrer
python manage.py import_poems ../corpus-json/ --dry-run
```

Règles appliquées à chaque import :

- **Publication automatique.** Un poème sans drapeau rouge est publié tout de suite et apparaît sur le site. Avec `CORPUS_AUTO_PUBLISH=0`, les poèmes attendent une publication manuelle dans l'administration.
- **Import répétable sans risque.** Un poème dont le texte n'a pas changé n'est pas réécrit ; ses informations (nom, acrostiche) sont seulement mises à jour si une règle a évolué.
- **Corrections préservées.** Les transcriptions corrigées à la main sont conservées quand un poème est réimporté.
- **Tout ou rien.** Chaque poème est enregistré dans une transaction : en cas d'erreur, rien n'est modifié.

6. La transcription en caractères latins

6.1 Pourquoi elle est générée automatiquement

Beaucoup de lecteurs, au Sénégal comme ailleurs, ne lisent pas l'écriture arabe mais lisent les caractères latins. Comme chaque lettre du corpus porte sa vocalisation complète, la transcription peut être produite par des règles, sans la saisir à la main. Les règles suivent la prononciation lors de la récitation, et non la simple correspondance lettre à lettre.

6.2 La convention locale retenue

La première convention suit l'orthographe à la française en usage au Sénégal. Elle a été validée par l'équipe :

Arabe	Latin	Exemple
ج	j	jamīl
ذ ظ ز	z	zâlika, zouhoûri
ث س ص	s (ss entre deux voyelles)	samanou, Rassoûl, hissâb
ش	ch	chaytân
خ	kh	khadîm
ق	q	qour'ân
ع	' (toujours écrit)	'alâ
voyelles longues	â î ôû	rahmân, rahîm, choukoûr

6.3 Les règles de prononciation

- **Liaison de l'article :** sa consonne se lie au mot précédent ; مِنَ الْكِتَابِ se lit *minal kitâbi*.
- **Lettres solaires :** la consonne est doublée ; مِنَ الشَّيْطَانِ se lit *minach chaytâni*.
- **Le nom d'Allah :** *billâhi, lillâhi, fadlou llâhi*.
- **Shadda, tanwin, ta marbuta :** consonne doublée ; *an, in, oun* ; t suivi de sa voyelle, muet à la pause (*rahma*).
- **Chaque hémistiché** commence une nouvelle respiration : son premier mot n'est pas lié au précédent.

Exemple complet :

Arabe	Transcription
أَعُوذُ بِاللَّهِ مِنَ الشَّيْطَانِ الرَّجِيمِ	a'oụzou billâhi minach chaytânir rajîmi
إِلَيَّ وَحْدِي سَلَبَ الْقُرْآنَا	ilayya wahdî salabal qour'ânâ

6.4 Stockage et corrections

- La transcription de chaque ligne est enregistrée dans `LineTranscription` à chaque import.
- Quelques mots (noms propres, lectures particulières) demandent une correction : dans l'administration, *Line transcriptions*, on modifie le texte et on enregistre. La ligne est alors marquée « corrigée à la main » et ne sera plus jamais remplacée automatiquement.
- Si l'arabe d'une ligne corrigée change, la correction est gardée mais signalée « à revoir ».
- Après une amélioration des règles : `python manage.py import_poems ... --refresh-transcriptions` régénère uniquement les transcriptions automatiques.
- Un **style anglais** sera ajouté ensuite comme second choix : même code, autre table de correspondance.

7. L'API REST

L'API est en lecture seule et ne montre que les poèmes publiés. Le site web et l'application mobile l'utilisent de la même façon.

Adresse (GET)	Contenu
<code>/api/v1/corpus/</code>	Totaux du corpus et les 7 diwans
<code>/api/v1/diwans/</code>	Les 7 diwans et leur nombre de poèmes en ligne
<code>/api/v1/diwans/diwan-01/</code>	Un diwan et la liste de ses khassaïdes publiées
<code>/api/v1/diwans/diwan-01/poems/008/</code>	Une khassida : lignes, transcriptions, poèmes précédent et suivant

Extrait de la réponse pour une khassida :

GET /api/v1/diwans/diwan-01/poems/008/

```
{
  "code": "D01K08", "number": 8, "slug": "008",
  "title": "...", "title_source": "name_line",
  "is_acrostic": true, "acrostic_match": 100, "bayt_count": 38,
  "diwan": { "number": 1, "slug": "diwan-01", "title": "..." },
  "lines": [
    { "position": 3, "section": "matn", "kind": "bayt", "bayt_number": 1,
      "hemistichs": ["...", "..."],
      "transcription": { "local": ["ilayya wahdî salabal qour'ânâ",
        "man kawnouhou lî bil mounâ qad ânâ"] } }
  ],
  "previous": { "slug": "002", "title": "...", "number": 2 },
  "next": null
}
```

Les textes sont envoyés en UTF-8. Une application qui lit l'API doit décoder les réponses en UTF-8 pour afficher correctement l'arabe (voir la partie 11).

8. Le site web (Angular)

8.1 Les pages

Adresse	Page	Données
/	Cheikh Ahmadou Bamba, le projet, le corpus en chiffres	Chiffres fixes + poèmes en ligne (API)
/diwans	Les 7 diwans et leurs khassaïdes en ligne	/api/v1/diwans/
/diwans/diwan-01	Un diwan : ses khassaïdes, recherche par nom	/api/v1/diwans/diwan-01/
/diwans/diwan-01/008	Une khassida : texte, acrostiche, transcription, poème précédent et suivant	/api/v1/diwans/diwan-01/poems/008/

Le menu principal donne accès à la biographie, à la liste déroulante des sept diwans (par leur titre arabe), au projet et au corpus. Sur téléphone, il se replie dans un menu.

8.2 Le rendu côté serveur

Chaque page est construite par le serveur Node.js d'Angular au moment de la visite, à partir des données de l'API. Deux conséquences : les moteurs de recherche reçoivent le texte arabe complet de chaque khassida, et un poème importé dans la base apparaît aussitôt sur le site, sans reconstruire ni redéployer le site. Pendant ce rendu, le serveur Angular appelle Django directement (variable `API_URL`) ; dans le navigateur, les appels passent par l'adresse `/api/v1`.

8.3 Identité visuelle

Élément	Choix	Raison
Couleurs	Vert profond #0F5A44, or #A3803A, fond clair #F3F4F0, encre #15211D	Sobriété et respect ; l'or rappelle l'enluminure et reste réservé aux fins ornements
Polices	Amiri (arabe et titres), Source Sans 3 (texte)	Amiri gère parfaitement la vocalisation complète ; Source Sans 3 est très lisible en français
Photographie	Dans un cadre en arc, avec un filet doré	L'arc évoque le mihrab ; c'est l'élément marquant de la page d'accueil
Chiffres du corpus	Tableau par diwan avec une barre de proportion	Montre la place de chaque diwan d'un coup d'œil
Page d'une khassida	Texte sur une « feuille », numéros de bayt à droite	Rappelle la page d'un diwan imprimé ; l'arabe se lit de droite à gauche

La biographie est rédigée d'après l'article « Ahmadou Bamba » de Wikipédia, cité avec sa licence (CC BY-SA 4.0) en bas de page.

8.4 Organisation du code

```

frontend/src/app/
├─ app.routes.ts           les pages et leurs adresses
├─ app.routes.server.ts   rendu côté serveur pour toutes les pages
├─ app.config.server.ts   adresse de l'API pendant le rendu serveur (API_URL)
├─ core/
│   ├─ api.ts             adresse de l'API, états chargement / prêt / erreur
│   ├─ arabic.ts          recherche sans vocalisation, lettres d'acrostiche
│   ├─ models/api.model.ts forme des données reçues de l'API
│   ├─ services/diwan-api.service.ts
│   └─ data/corpus-stats.ts titres et chiffres des 7 diwans
├─ shared/                en-tête et menu, pied de page, fil d'Ariane, bayt, page 404
└─ features/
    ├─ home/              accueil : présentation, biographie, projet, corpus
    ├─ diwans/diwan-list/ /diwans
    ├─ diwans/diwan-detail/ /diwans/:diwan
    └─ poem/poem-page/    /diwans/:diwan/:poem

```

8.5 Détails de réalisation

- **Recherche sans harakat** : la liste des khassaïdes d'un diwan se filtre avec la même normalisation que le serveur.
- **Lettres d'acrostiche en rouge** : un caractère invisible de liaison (ZWJ) garde la lettre colorée attachée au reste du mot.
- **Transcription mémorisée** : le choix « afficher la transcription » est gardé d'un poème à l'autre dans le navigateur.
- **Polices hébergées par le site** (paquets Fontsource) : pas de dépendance à Google, affichage plus rapide avec une connexion lente.
- **En développement**, `npm start` redirige `/api` vers Django (fichier `proxy.conf.json`).

9. Installer et lancer le projet

9.1 Prérequis

Outil	Version	Utilisé par
Git	récente	tous
Python	3.12 ou plus	backend, outil « Poem to JSON »
PostgreSQL	16 ou plus	backend
Node.js	22.12 ou plus	frontend
Flutter	stable récente	mobile

9.2 Le backend

```
cd diwan-platform/backend
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # macOS / Linux
pip install -r requirements.txt

# PostgreSQL : base "diwan", utilisateur "diwan"
# (ou variables DB_NAME, DB_USER, DB_PASSWORD, DB_HOST)
python manage.py migrate
python manage.py seed_diwans
python manage.py createsuperuser
python manage.py runserver
# administration : http://127.0.0.1:8000/admin/
```

9.3 Le site web

```
cd diwan-platform/frontend
npm install
npm start                      # http://localhost:4200 (Django doit tourner)
```

9.4 Windows, macOS et Linux

Le projet fonctionne sur les trois systèmes. Deux points ont été traités explicitement :

- **Administration sans mise en forme.** Sur certains PC Windows, le registre déclare les fichiers `.css` et `.js` comme du texte brut ; le navigateur refuse alors les feuilles de style et l'administration apparaît sans mise en forme. Le correctif est inclus dans `settings.py` : les types corrects sont imposés au démarrage, et **WhiteNoise** sert ces fichiers avec sa propre table de types, indépendante du système. Il fonctionne aussi sous macOS et Linux, testé avec DEBUG activé et désactivé.
- **Encodage de l'arabe.** Windows lit les fichiers texte dans son encodage local s'il n'est pas précisé. Tous les fichiers lus ou écrits par le projet précisent l'UTF-8 : l'arabe se comporte de la même façon sur les trois systèmes.
- Sous macOS, la commande est `python3`, et PostgreSQL s'installe facilement avec Postgres.app ou Homebrew.

10. Travailler à deux avec Git et GitHub

10.1 Principe

L'équipe partage **un seul dépôt GitHub** contenant `backend/`, `frontend/`, `mobile/` et `corpus-json/`. Personne ne s'envoie plus le code par fichier `.zip` : chacun récupère le dépôt, travaille sur **une branche**, puis propose ses changements par une **pull request** que l'autre relit avant de les intégrer à la branche principale `main`. Chaque fusion dans `main` déclenche le redéploiement automatique du service concerné (partie 12).

Rôle	Travaille dans	Ne touche pas à
Responsable backend	backend/, corpus-json/, docs/, tools/	frontend/ sans accord préalable
Responsable frontend	frontend/ (et mobile/ plus tard)	backend/ ; il ouvre une <i>issue</i> pour demander un champ ou une adresse d'API

Une seule base de données existe : celle de la plateforme en ligne. Le responsable backend est le seul à y importer et à y publier des poèmes ; le responsable frontend la lit à travers l'API, en lecture seule.

10.2 Création du dépôt (responsable backend, une seule fois)

1. Installer Git et configurer son identité :

```
git config --global user.name "Prénom Nom"
git config --global user.email "adresse@exemple.com"
```

2. Rassembler le projet dans un dossier `diwan-platform` contenant `backend/` et `frontend/` (le code actuel, **sans** les dossiers `node_modules`, `.env`, `dist`).
3. Créer à la racine le fichier `.gitignore`, qui empêche d'envoyer sur GitHub les fichiers lourds ou secrets :

.gitignore

```
# Python / Django
__pycache__/
*.pyc
.venv/
backend/media/
backend/staticfiles/
.env

# Angular
frontend/node_modules/
frontend/dist/
frontend/.angular/

# Flutter
mobile/build/
mobile/.dart_tool/

# Systèmes
.DS_Store
Thumbs.db
.vscode/
.idea/
```

4. Créer à la racine le fichier `.gitattributes`, qui évite les conflits de fins de ligne entre Windows et macOS :

.gitattributes

```
* text=auto
*.docx binary
*.png binary
*.jpg binary
```

5. Premier enregistrement (commit) :

```
cd diwan-platform
git init -b main
git add .
git commit -m "Premier commit : backend Django et frontend Angular"
```

6. Sur github.com : **New repository**, nom **diwan-platform**, visibilité **Private**, sans README ni .gitignore (ils existent déjà).

7. Relier le dossier au dépôt et envoyer le code :

```
git remote add origin https://github.com/<compte>/diwan-platform.git
git push -u origin main
```

Au premier envoi, Git ouvre une fenêtre de connexion à GitHub (Git Credential Manager, installé avec Git pour Windows). Sous macOS, le plus simple est d'installer GitHub CLI et de lancer `gh auth login`.

- Inviter le collaborateur : **Settings > Collaborators > Add people**, avec son identifiant GitHub.
- Protéger `main` : **Settings > Branches** (ou **Rules > Rulesets**), règle sur `main` : *Require a pull request before merging*. Plus personne ne peut modifier `main` directement.

10.3 Première installation (responsable frontend)

Le collaborateur **ne travaille pas à partir d'un .zip reçu** : il clone le dépôt, qui contient toujours la dernière version commune.

- Accepter l'invitation reçue par e-mail, installer Git, Node.js 22.12+, Python 3.12+ et PostgreSQL.
- Récupérer le projet :

```
git clone https://github.com/<compte>/diwan-platform.git
cd diwan-platform
```

- Lancer le backend en local (partie 9.2) pour disposer de l'API, puis le site (partie 9.3). Pour avoir des poèmes à afficher : `python manage.py import_poems ../corpus-json/`.

S'il a déjà modifié le code reçu en .zip

Ses modifications ne sont pas perdues. Il clone le dépôt, crée une branche, copie par-dessus les fichiers qu'il a changés (jamais `node_modules`), vérifie, puis enregistre :

```
git switch -c frontend/reprise-modifications
# copier ses fichiers modifiés dans frontend/
git status      # liste ce qui a changé
git add frontend
git commit -m "Frontend : reprise des modifications faites hors du dépôt"
git push -u origin frontend/reprise-modifications
```

Il ouvre ensuite une pull request (partie 10.4).

10.4 Le cycle de travail quotidien

1. Partir de la version commune à jour :

```
git switch main
git pull
```

2. Créer une branche pour une tâche précise (préfixe `backend/`, `frontend/` ou `mobile/`) :

```
git switch -c frontend/page-recherche
```

3. Travailler, puis enregistrer par petites étapes avec des messages clairs :

```
git add frontend
git commit -m "Frontend : champ de recherche sur la page d'un diwan"
```

4. Envoyer la branche sur GitHub :

```
git push -u origin frontend/page-recherche
```

5. Sur GitHub, **Compare & pull request** : décrire le changement, désigner l'autre membre comme relecteur.
6. Le relecteur lit les modifications, commente ou approuve ; l'auteur clique sur **Merge pull request**, puis supprime la branche.
7. Chacun récupère la nouvelle version : `git switch main` puis `git pull`.

10.5 Règles pour éviter les conflits

- **Chacun son dossier.** Le backend modifie `backend/`, le frontend modifie `frontend/`. Deux personnes qui modifient des dossiers différents n'ont jamais de conflit.
- **L'API est le contrat.** Le frontend ne dépend que des adresses et des champs de l'API. Tout changement d'API est annoncé dans la pull request et noté dans `docs/api.md`, puis relu par le responsable frontend.
- **Les migrations sont créées par le responsable backend uniquement.** Le collaborateur les applique avec `python manage.py migrate` après chaque `git pull`.
- **On ne partage jamais la base elle-même.** Chacun a sa base locale ; les données communes passent par le code (migrations, `seed_diwans`) et par les fichiers `corpus-json/`, importés avec `import_poems`.
- **Pas de secrets dans le dépôt :** mots de passe et clés restent dans des variables d'environnement.
- **Les fichiers Word sources** (volumineux, binaires) restent dans un dossier partagé (Google Drive par exemple) ; seul leur JSON relu est versionné.
- **Mettre sa branche à jour** si `main` a avancé pendant son travail : `git switch ma-branche` puis `git merge main`.

10.6 Résoudre un conflit

1. Git signale les fichiers en conflit après `git merge main` ou `git pull`.
2. Ouvrir chaque fichier : les passages concernés sont encadrés par `<<<<<<`, `=====` et `>>>>>>`.

3. Garder la bonne version (ou combiner les deux), supprimer les marqueurs. VS Code propose des boutons *Accept Current* / *Accept Incoming* / *Accept Both*.
4. Enregistrer : `git add fichier` puis `git commit`.

Pour suivre les tâches, les **Issues** de GitHub (une par tâche, attribuée à une personne) et un tableau **Projects** (À faire / En cours / Terminé) suffisent.

10.7 Côté backend : faire passer une modification dans le dépôt

À faire après chaque séance de relecture ou de développement :

1. Se placer sur la version commune à jour :

```
git switch main
git pull
```

2. Créer une branche décrivant la tâche :

```
git switch -c backend/import-diwan-01
```

3. Travailler : importer ou corriger des poèmes dans l'administration en ligne, modifier le code, etc.
4. Après un import ou une correction, exporter le corpus pour que l'équipe l'ait aussi :

```
cd backend
python manage.py export_poems      # écrit ../corpus-json/diwan-01/D01K08.json ...
```

5. Enregistrer et envoyer :

```
git add backend corpus-json
git commit -m "Backend : import du diwan 1"
git push -u origin backend/import-diwan-01
```

6. Sur GitHub : **Compare & pull request**, désigner le collaborateur comme relecteur, puis fusionner. Rendre redéploie l'API automatiquement.

Ce qui n'est jamais dans le dépôt

- La base de données elle-même : elle vit chez l'hébergeur PostgreSQL. Le dépôt contient les poèmes au format JSON, qui permettent de la reconstruire n'importe où.
- Les mots de passe, la chaîne de connexion et la clé secrète : ils restent dans les variables d'environnement de l'hébergeur.

10.8 Côté frontend : faire passer une modification dans le dépôt

Le responsable frontend n'installe ni Python ni PostgreSQL : il lit les poèmes par l'API en ligne.

1. Installation, une seule fois :

```
git clone https://github.com/<compte>/diwan-platform.git
cd diwan-platform/frontend
npm install
```

2. Indiquer l'adresse de l'API en ligne dans `proxy.conf.deployed.json` :

`frontend/proxy.conf.deployed.json`

```
{ "/api": { "target": "https://diwan-platform.onrender.com",
            "secure": true, "changeOrigin": true } }
```

3. Travailler chaque jour :

```
git switch main && git pull
git switch -c frontend/page-recherche
npm run start:api # http://localhost:4200, avec les vrais poèmes
# ... modifications ...
git add frontend
git commit -m "Frontend : champ de recherche sur la page d'un diwan"
git push -u origin frontend/page-recherche
```

4. Ouvrir la pull request sur GitHub. Après la fusion, Render reconstruit le site en ligne.

Si l'API est momentanément indisponible, il peut travailler hors ligne : `npm start` avec un backend local rempli par `python manage.py import_poems ../corpus-json/`.

11. L'application mobile Flutter

11.1 Principe

L'application utilise **le même backend et la même base de données** que le site, en passant par **la même API**. Elle ne se connecte jamais directement à PostgreSQL. Un poème importé est donc visible en même temps sur le site et dans l'application, sans travail supplémentaire.

```
PostgreSQL ← Django (API /api/v1)
                |
                ├── site web Angular
                └── application Flutter
```

11.2 Création dans le dépôt

L'application est un troisième dossier du même dépôt, créé sur une branche `mobile/` :

```
cd diwan-platform
git switch -c mobile/creation-application
flutter create --org sn.diwan --project-name diwan_app mobile
cd mobile
flutter pub add http go_router
flutter run
```

11.3 Organisation du code

```
mobile/lib/
├─ main.dart
├─ core/
│   ├─ config.dart           adresse de l'API
│   └─ api_client.dart       appels à /api/v1
├─ data/
│   └─ models/               diwan.dart, poem.dart, poem_line.dart (champs de l'API)
├─ features/
│   ├─ home/                 accueil
│   ├─ diwans/               liste des diwans, page d'un diwan
│   └─ poem/                 page d'une khassida, transcription
└─ shared/widgets/
    └─ bayt_view.dart         un bayt : hémistiches et transcription
```

Les écrans suivent les mêmes pages que le site : liste des diwans, khassaïdes d'un diwan, texte d'une khassida avec transcription. Les modèles Dart reprennent exactement les champs de l'API (partie 7).

11.4 Premier appel à l'API

lib/core/api_client.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class DiwanSummary {
  DiwanSummary({
    required this.number,
    required this.slug,
    required this.title,
    required this.poemCount,
  });
  final int number;
  final String slug;
  final String title;
  final int poemCount;

  factory DiwanSummary.fromJson(Map<String, dynamic> json) => DiwanSummary(
    number: json['number'] as int,
    slug: json['slug'] as String,
    title: json['title'] as String,
    poemCount: json['poem_count'] as int,
  );
}

class ApiClient {
  ApiClient(this.baseUrl);
  final String baseUrl;

  Future<List<DiwanSummary>> getDiwans() async {
    final response = await http.get(Uri.parse('$baseUrl/diwans/'));
    if (response.statusCode != 200) {
      throw Exception('Erreur API ' + response.statusCode.toString());
    }
    // décoder en UTF-8, sinon l'arabe s'affiche mal
    final data = jsonDecode(utf8.decode(response.bodyBytes)) as List<dynamic>;
    return data
      .map((e) => DiwanSummary.fromJson(e as Map<String, dynamic>))
      .toList();
  }
}
```

11.5 Adresse de l'API pendant le développement

Appareil	Adresse à utiliser	À prévoir
Émulateur Android	http://10.0.2.2:8000/api/v1	Autoriser le HTTP en développement (android:usesCleartextTraffic)
Simulateur iOS	http://127.0.0.1:8000/api/v1	NSAllowsLocalNetworking dans Info.plist
Téléphone réel (même Wi-Fi)	http://<adresse IP du PC>:8000/api/v1	runserver 0.0.0.0:8000 et l'adresse IP ajoutée à DJANGO_ALLOWED_HOSTS
Production	https://<domaine>/api/v1	HTTPS obligatoire

11.6 Plus tard

- **Lecture hors connexion** : enregistrer les poèmes consultés dans une base locale (sqflite ou drift).
- **Police Amiri** intégrée à l'application pour un affichage identique au site.
- **Stabilité de l'API** : les adresses `/api/v1` ne changent pas de forme ; une évolution incompatible passerait par `/api/v2`, pour ne jamais casser les applications déjà installées.

12. La plateforme en ligne

12.1 Les deux services et la base

La plateforme tourne en trois morceaux, hébergés gratuitement pendant le développement :

Élément	Hébergeur	Rôle
diwan-platform	Render (service web, Python)	L'API <code>/api/v1/...</code> et l'administration <code>/admin/</code>
diwan-web	Render (service web, Node)	Le site, rendu côté serveur, qui lit l'API
La base PostgreSQL	Neon (ou Supabase)	Les poèmes : la seule base du projet

Le plan gratuit de Render met les services en veille après quinze minutes sans visite ; la première visite suivante prend environ une minute. La base PostgreSQL de Render expire au bout de trente jours, celle de Neon non : c'est pourquoi la base est chez Neon. Pour le lancement officiel, l'équipe passera sur un hébergement payant.

12.2 Le service de l'API (Django)

Champ Render	Valeur
Language	Python 3
Root Directory	backend
Build Command	bash build.sh
Start Command	gunicorn config.wsgi:application --bind 0.0.0.0:\$PORT
Health Check Path	/api/v1/corpus/

Variables d'environnement :

Variable	Valeur
DATABASE_URL	La chaîne de connexion donnée par Neon
DJANGO_SECRET_KEY	Une longue chaîne aléatoire (<code>python -c "import secrets; print(secrets.token_urlsafe(64))"</code>)
DJANGO_DEBUG	0
DJANGO_ALLOWED_HOSTS	diwan-platform.onrender.com (sans <code>https://</code>)
DJANGO_CSRF_TRUSTED_ORIGINS	<code>https://diwan-platform.onrender.com</code>
PYTHON_VERSION	3.12.6
CORPUS_AUTO_PUBLISH	0 pendant la relecture, 1 ensuite

`build.sh` installe les dépendances, rassemble les fichiers de l'administration (`collectstatic`), applique les migrations, crée les sept diwans (`seed_diwans`) et, si les variables `DJANGO_SUPERUSER_*` sont présentes, crée le compte administrateur (`ensure_superuser`) — utile car le plan gratuit n'offre pas de console.

12.3 Le service du site (Angular)

Champ Render	Valeur
Language	Node
Root Directory	frontend
Build Command	<code>npm ci npm install; npm run build</code>
Start Command	<code>node dist/diwan-web/server/server.mjs</code>
API_URL	<code>https://diwan-platform.onrender.com/api/v1</code> (rendu côté serveur)
NG_ALLOWED_HOSTS	<code>diwan-web.onrender.com</code> (Angular refuse les autres adresses)
NODE_VERSION	22.12.0

L'adresse de l'API utilisée par le **navigateur** est fixée à la construction, dans `angular.json` (configurations > production > define > `NG_API_BASE_URL`). Le fichier `proxy.conf.deployed.json` sert au développement local ; il n'intervient pas en ligne.

12.4 Alimenter la base en ligne

Depuis son propre ordinateur, le responsable backend travaille sur la base en ligne en donnant simplement son adresse :

```
cd backend
set "DATABASE_URL=<chaîne Neon>"          REM Windows (cmd)
$env:DATABASE_URL="<chaîne Neon>"         # Windows (PowerShell)
export DATABASE_URL="<chaîne Neon>"        # macOS / Linux

python manage.py createsuperuser           # le compte d'administration en ligne
python manage.py import_poems ../corpus-json/
```

Refermer ensuite la fenêtre du terminal (ou effacer la variable) pour ne pas continuer à travailler sur la base en ligne sans s'en rendre compte. Vérification utile avant tout import :

```
python manage.py shell -c "from django.db import connection; print(connection.settings_dict['HOST'])"
```

12.5 Sécurité et confidentialité

- **L'API est en lecture seule** : elle ne montre que les poèmes publiés et n'accepte aucune modification. Seule l'administration, protégée par mot de passe, permet d'importer et de publier.
- **Le collaborateur ne reçoit jamais** la chaîne de connexion de la base ni un compte administrateur : il travaille avec l'adresse de l'API.

- **Le site n'est pas référencé** avant le lancement : un fichier `robots.txt` (site et API) et une balise `noindex` demandent aux moteurs de recherche de ne pas l'indexer. Les deux fichiers portent en commentaire ce qu'il faudra changer le jour du lancement.
- **Les secrets ne sont jamais dans le dépôt** : ils vivent dans les variables d'environnement de l'hébergeur.

12.6 Problèmes rencontrés et leur cause

Message	Cause	Correction
connection to 127.0.0.1:5432 refused pendant le déploiement	DATABASE_URL absente : Django retombait sur la base locale	Ajouter la variable ; <code>settings.py</code> affiche désormais un message clair si elle manque
Not Found à la racine du service API	Le backend ne servait que <code>/api/v1/</code> et <code>/admin/</code>	Une page d'index a été ajoutée à la racine
<code>npm ci : package.json and package-lock.json are not in sync</code>	Le fichier <code>package-lock.json</code> ne contenait pas toutes les dépendances	<code>npm install</code> puis commit du fichier ; la commande de build tolère les deux cas
Site rendu vide, <i>Header host is not allowed</i>	Angular refuse de rendre une page pour une adresse inconnue	Variable <code>NG_ALLOWED_HOSTS</code> , lue dans <code>src/server.ts</code>
Administration sans mise en forme (Windows)	Le registre déclare <code>.css</code> et <code>.js</code> comme texte brut	Types imposés dans <code>settings.py</code> + WhiteNoise (partie 9.4)

13. Prochaines étapes

- Importer les khassaïdes relues, diwan par diwan.
- Classements par mois de l'année hégirienne (de Muharram à Dhou al-Hijja), par événement (Mawlid, Achoura...) et par jour, à partir des noms des poèmes.
- Recherche dans tout le corpus : d'abord les noms des poèmes, puis les abyat.
- Génération des fichiers PDF (poème, diwan, corpus) et bouton de téléchargement.
- Style de transcription anglais, proposé comme second choix.
- Application mobile Flutter sur la même API.
- Lancement officiel : hébergement payant (API, site et base), nom de domaine, `robots.txt` ouvert, plan du site pour l'indexation des khassaïdes.
- Formes wolofales sans version courte : les dessiner, puis relancer l'outil d'espacement (voir `docs/polices.md`).